

AUIPivotReact.tsx 서브 컴포넌트 설명서

AUIPivotReact.tsx는 React와 TypeScript에서 AUIPivot을 사용할 수 있도록 제공하는 서브 컴포넌트입니다.

1. 정의 가능한 속성들

속성명	유형	기본값	설명
autoResize	Boolean	true	자동 크기 감지 사용 여부입니다.
resizeMode	String	window	window: 창 크기 감지 container: 호스트 DIV 크기 감지
resizeDelayTime	Number	300	마지막 알림 후 대기 시간(ms)입니다.
createOnMounted	Boolean	true	마운트 시 피벗을 자동 생성합니다.
pivotProps	IPivot.Props	{}	피벗 엔진의 생성 속성입니다.
name	String	"	호스트 DIV의 ID에 사용할 이름입니다.
waitPortalRendering	Boolean	false	Portal 영역 준비를 위해 생성을 한 프레임 늦춥니다.

name은 선택 사항입니다. 생략하면 임의 ID를 생성합니다. name='showcase01'이면 호스트 DIV의 ID는 aui-pivot-wrap-showcase01입니다. 같은 화면의 name은 서로 다르게 지정하세요.

컴포넌트의 getPID()로 피벗 ID를 확인할 수 있습니다. DOM 구조는 다음과 같습니다.

```
<div id="aui-pivot-wrap-showcase01" style="position: relative; height: 560px;">  
  <div id="aui-pivot-wrap-showcase01__aui_pivot_filter_panel" class="aui-pivot-fi  
  <div class="aui-pivot" style="position: relative; height: 560px;">
```

2. 피벗에 접근하여 메소드 사용하기

useRef로 컴포넌트를 참조합니다. 마운트 이후 ref에서 메소드를 호출합니다.

```
// 피벗 컴포넌트 참조
const myPivot = useRef<AUIPivot | null>(null);
```

생성 태그에 ref를 연결합니다.

```
<AUIPivot ref={myPivot} />
```

인스턴스 ID는 컴포넌트가 전달하므로 메소드 인수에 넣지 않습니다.

```
const pivot = myPivot.current;
if (pivot) pivot.setRowFields(['REGION', 'NAME']);
```

createOnMounted=false로 지정한 경우 마운트 이후 create(pivotProps)를 호출합니다. destroy()

후에는 create()로 다시 만들고 필드와 데이터를 설정합니다.

컴포넌트가 해제될 때 엔진과 예약 작업을 정리합니다. 비동기 데이터 요청이 해제 후 완료되면 ref와 isCreated()를 확인한 뒤 데이터를 전달하세요.

React는 bind()로 이벤트를 연결합니다.

TypeScript 타입 패키지

npm install aui-pivot@latest로 타입 패키지를 설치합니다. 타입과 EventKind를 제공하며 엔진과 라이선스는 포함하지 않습니다. 제품 파일은 원하는 위치에 두고 컴포넌트 상단의 import 경로를 맞추세요.

import type * as IPivot from 'aui-pivot'; 형식으로 Props와 이벤트 타입을 사용합니다. lock 파일로 설치 버전을 고정할 수 있고 갱신은 npm update aui-pivot으로 합니다.

React ref는 AUIPivot<Row>로 원본 행 타입도 지정할 수 있습니다. 새 resizeMode 옵션은 컴포넌트가 선언하므로 기존 npm 타입 패키지에서도 사용할 수 있습니다.

3. React에서 AUIPivotReact.tsx 사용하기

```
import { useEffect, useRef } from 'react';
import AUIPivot from '@static/AUIPivot-React.tsx/AUIPivotReact';
import type * as IPivot from 'aui-pivot';

export default function Report() {
  const pivotProps: IPivot.Props = { height: 400 };
  const myPivot = useRef<AUIPivot | null>(null);
  // 시작점에서 피벗을 생성한 뒤 자동차 자료를 읽습니다.
  function init() {
    createPivotGrid();
    return requestData(`${import.meta.env.BASE_URL}data/car_sales.json`);
  }
  function createPivotGrid() {
    const pivot = myPivot.current;
    if (!pivot) return;
    pivot.create(pivotProps);
    pivot.setRowFields(['REGION', 'NAME']);
    pivot.setColumnFields(['COLOR']);
    pivot.setValueFields([
      { dataField: 'TOTAL', operation: 'SUM' }
    ]);
    pivot.bind('pivotComplete', event => console.log(event.type));
  }
  // 요청 도중 화면이 해제되면 결과를 사용하지 않습니다.
  function requestData(url: string) {
    let disposed = false;
    fetch(url).then(response => {
      if (!response.ok) throw new Error('자료 요청 실패');
      return response.json();
    }).then(data => {
      const pivot = myPivot.current;
      if (!disposed && pivot?.isCreated()) pivot.setGridData(data);
    }).catch(error => { if (!disposed) console.error(error); });
    return () => { disposed = true; };
  }
  useEffect(() => init(), []);
  return <AUIPivot ref={myPivot} createOnMounted={false}
    resizeMode="container" resizeDelayTime={100} />;
}
```

4. 컨테이너 크기 감지 사용하기

`resizeMode="container"`를 지정하면 `ResizeObserver`로 피벗을 담는 호스트 `DIV`의 크기를 감지합니다. 창 크기가 그대로여도 사이드바 접기나 부모 배치 변경에 따라 크기를 조정합니다.

옵션은 `pivotProps` 내부가 아닌 `AUIPivot` 컴포넌트에 직접 지정합니다. 앞 절 예제는 100ms 지연 후 조정하도록 설정했습니다.

기본값과 호출 시점

`resizeMode` 기본값은 `"window"`입니다. 생략하면 기존 창 크기 이벤트를 사용합니다. `autoResize` 기본값은 `true`이며 `false`로 지정하면 자동 감지를 등록하지 않습니다.

`resizeDelayTime` 기본값은 300ms입니다. `"container"`는 마지막 알림 이후 지정 시간만큼 기다리고 다음 애니메이션 프레임에 반영합니다. 0으로 지정해도 동기 실행하지 않습니다.

DOM이 분리되었거나 너비 또는 높이가 5px 이하이면 호출하지 않습니다. 크기가 같으면 생략하고, 숨겨진 영역이 다시 보이면 재측정합니다.

`resizeMode`와 `autoResize`는 생성 시 지정하세요. 실행 중 `prop` 변경만으로 감지 방식을 전환하지 않습니다. `ResizeObserver`가 없으면 창 크기 이벤트를 사용합니다.

레이아웃과 생명주기

너비를 부모에 맞추려면 `pivotProps`의 `width`를 생략합니다. 높이도 부모에 맞추려면 부모와 호스트 CSS에 유효한 높이를 지정하세요. `Flex/Grid`에서는 필요에 따라 `min-width: 0`을 지정하고 콘텐츠가 부모를 계속 늘리는 순환 배치를 피하세요.

컴포넌트를 해제하거나 `destroy()`를 호출하면 컨테이너 관찰자, 타이머, 애니메이션 프레임을 정리합니다. `create()`로 다시 만들면 관찰을 다시 연결합니다.

`React StrictMode`에서 해제된 예약은 이후에 생성된 피벗을 조정하지 않습니다. `Portal`에서 영역 준비를 기다려야 하면 `waitPortalRendering`을 사용하세요.

크기 변경을 위해 피벗이나 원본 데이터를 다시 만들지 않습니다. 수동 `resize()`의 전달 방식과 엔진의 데이터 기반 높이 정책은 유지합니다.